

SISTEMAS OPERATIVOS II

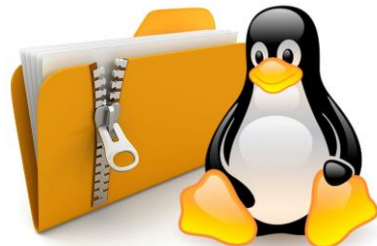
2026

TEMAS – UNIDAD 3

2

Entrada Salida / Sistema de Archivos

- Comprender la arquitectura de E/S en Linux.
- Analizar archivos especiales y drivers.
- Entender sockets y comunicación en red.
- Estudiar el sistema de archivos Linux.
- Analizar VFS, ext2, ext3, ext4 y NFS.



ADMINISTRACIÓN DE E/S EN LINUX

Filosofía

Principio fundamental:

"Todo dispositivo es tratado como un archivo"

Ventajas:

- Interfaz uniforme.
- Simplicidad para programadores.
- Reutilización de llamadas al sistema.

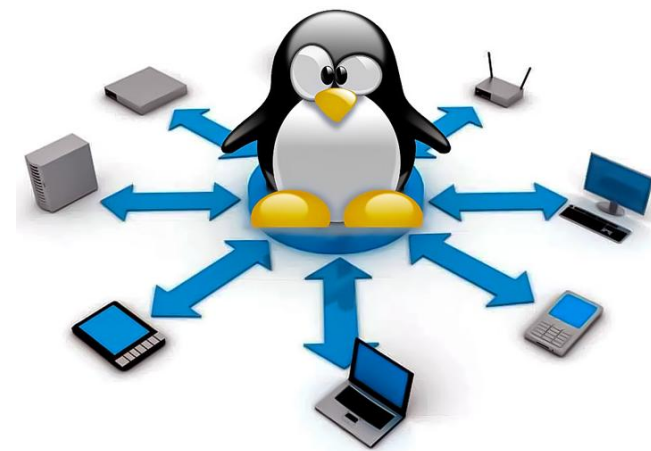
Llamadas utilizadas:

- `open()`
- `read()`
- `write()`
- `close()`

Ejemplo:

`cp informe.txt /dev/lp`

La aplicación no sabe que está escribiendo en una impresora.



ADMINISTRACIÓN DE E/S EN LINUX

4

Archivos Especiales

Linux representa dispositivos mediante archivos especiales ubicados normalmente en:

/dev

Ejemplos:

- /dev/tty
- /dev/lp
- /dev/hda

Archivos de bloques

- Acceso aleatorio.
- Utilizados por discos.

Archivos de caracteres

- Flujo secuencial.
- Teclados.
- Impresoras.
- Terminales.
- Mouse.



ADMINISTRACIÓN DE E/S EN LINUX

5

Dispositivos Mayor y Menor

Cada dispositivo posee:

- Número Mayor (Major)

Identifica al driver.

- Número Menor (Minor)

Identifica al dispositivo físico.

Ejemplo:

Driver Disco SATA

- └ Disco 1 (minor 0)
- └ Disco 2 (minor 1)
- └ Disco 3 (minor 2)

Permite que un mismo controlador maneje múltiples dispositivos.



ADMINISTRACIÓN DE E/S EN LINUX

6

Personalización de Terminales

Linux permite configurar:

- Tecla de borrado.
- CTRL-C.
- CTRL-U.
- Eco de caracteres.
- Conversión CR/LF.
- Expansión de tabuladores.

Esto se realiza mediante llamadas especiales de configuración.

Ventaja:

- Mayor flexibilidad para el usuario.



ADMINISTRACIÓN DE E/S EN LINUX

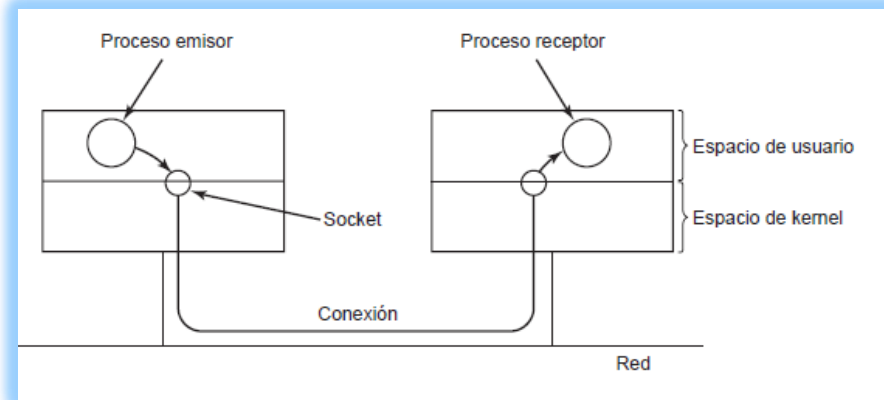
7

Redes y Sockets

Concepto central: Socket Interfaz entre una aplicación y la red.

Funciones:

- Crear conexión.
- Enviar datos.
- Recibir datos.
- Cerrar comunicación.



Al crearse devuelve un descriptor de archivo.

TIPOS

1. Flujo de bytes confiable

- Orientado a conexión.
- Garantiza entrega.
- Garantiza orden.

TCP

2. Flujo de paquetes confiable

Conserva límites de paquetes.

3. Datagramas no confiables

- Mayor velocidad.
- No garantiza entrega.

UDP

ADMINISTRACIÓN DE E/S EN LINUX

8

Llamadas al sistema

Llamada a función	Descripción
<code>s = cfsetospeed(&termios, velocidad)</code>	Establece la velocidad de salida
<code>s = cfsetispeed(&termios, velocidad)</code>	Establece la velocidad de entrada
<code>s = cfgetospeed(&termios, velocidad)</code>	Obtiene la velocidad de salida
<code>s = cfgetispeed(&termios, velocidad)</code>	Obtiene la velocidad de entrada
<code>s = tcsetattr(fd, opc, &termios)</code>	Establece los atributos
<code>s = tcgetattr(fd, &termios)</code>	Obtiene los atributos

Velocidad de Terminal

Establecer y leer de vuelta todos los caracteres especiales que se utilizan para borrar caracteres y líneas.



ADMINISTRACIÓN DE E/S EN LINUX

9

Drivers de Dispositivos

Objetivo:

Ocultar detalles del hardware.

Características:

- Ejecutan en modo kernel.
- Proveen interfaz estandarizada.
- Aislan dependencias del dispositivo.

Cada driver implementa:



- Open
- Close
- Read
- Write
- Ioctl



ADMINISTRACIÓN DE E/S EN LINUX

10

Cache de E/S

Objetivos: ➡ Reducir accesos al disco.
➡ Mejorar rendimiento.



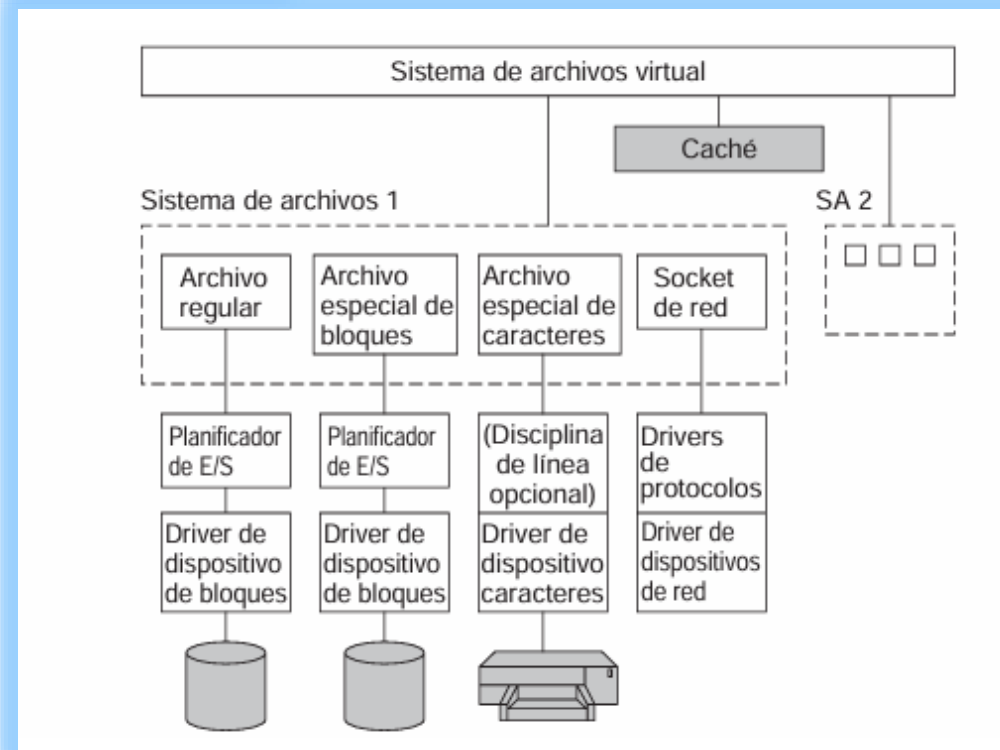
Proceso:

Buscar en caché.

- Si existe → usarla.
- Si no existe → leer disco.

Beneficios:

- Menor latencia.
- Menor uso de disco.



Demonio pdflush

Responsable de:

- Escribir páginas sucias.
- Liberar memoria.
- Sincronizar caché con disco.

Acciones:

- Escritura periódica.
- Escritura por presión de memoria.

En modo laptop:

Mayor frecuencia de escritura para ahorrar energía.

ADMINISTRACIÓN DE E/S EN LINUX

Planificador de E/S

Problema:

Mover la cabeza del disco es costoso.

Solución:

Elevator Scheduler

Ordena peticiones según:

- Dirección del sector.
- Cercanía física.

Ventajas:

- Menos movimientos.
- Mayor rendimiento.

Módulos cargables

Linux permite agregar código al kernel en ejecución.

Ventajas:

- No recompilar kernel.
- Flexibilidad.
- Fácil incorporación de hardware.

Ejemplos:

- Drivers.
- Sistemas de archivos.
- Protocolos de red.



SISTEMA DE ARCHIVOS LINUX



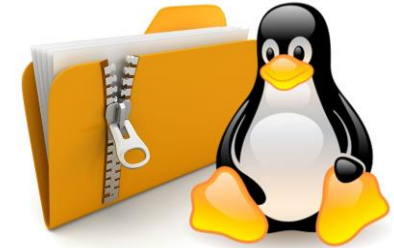
12

Objetivos

- Comprender la organización lógica del sistema de archivos Linux.
- Analizar rutas, directorios y enlaces.
- Estudiar las llamadas al sistema relacionadas con archivos.
- Comprender la arquitectura VFS.
- Analizar la implementación de ext2, ext3 y ext4
- Introducir NFS y sistemas de archivos distribuidos.

SISTEMA DE ARCHIVOS EN LINUX

13



Evolución de los Sistemas de Archivos en Linux

Sistema	Características
MINIX	Nombres de 14 caracteres
EXT	Primer sistema extendido
EXT2	Mejor rendimiento
EXT3	Journaling
Otros	FAT, NTFS, XFS, NFS, PROC

Ideas Claves

- Linux soporta múltiples sistemas de archivos.
- El VFS permite transparencia entre ellos.
- Los archivos son secuencias de bytes.

Qué es un archivo en Linux?

Características:

- Secuencia arbitraria de bytes.
- Sin distinción entre texto y binario.
- Nombres hasta 255 caracteres.
- Cualquier carácter ASCII excepto NUL.

Ejemplos:

- programa.c
- datos.dat
- imagen.jpg
- script.sh

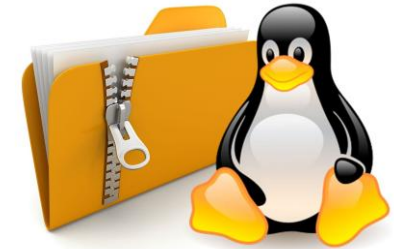
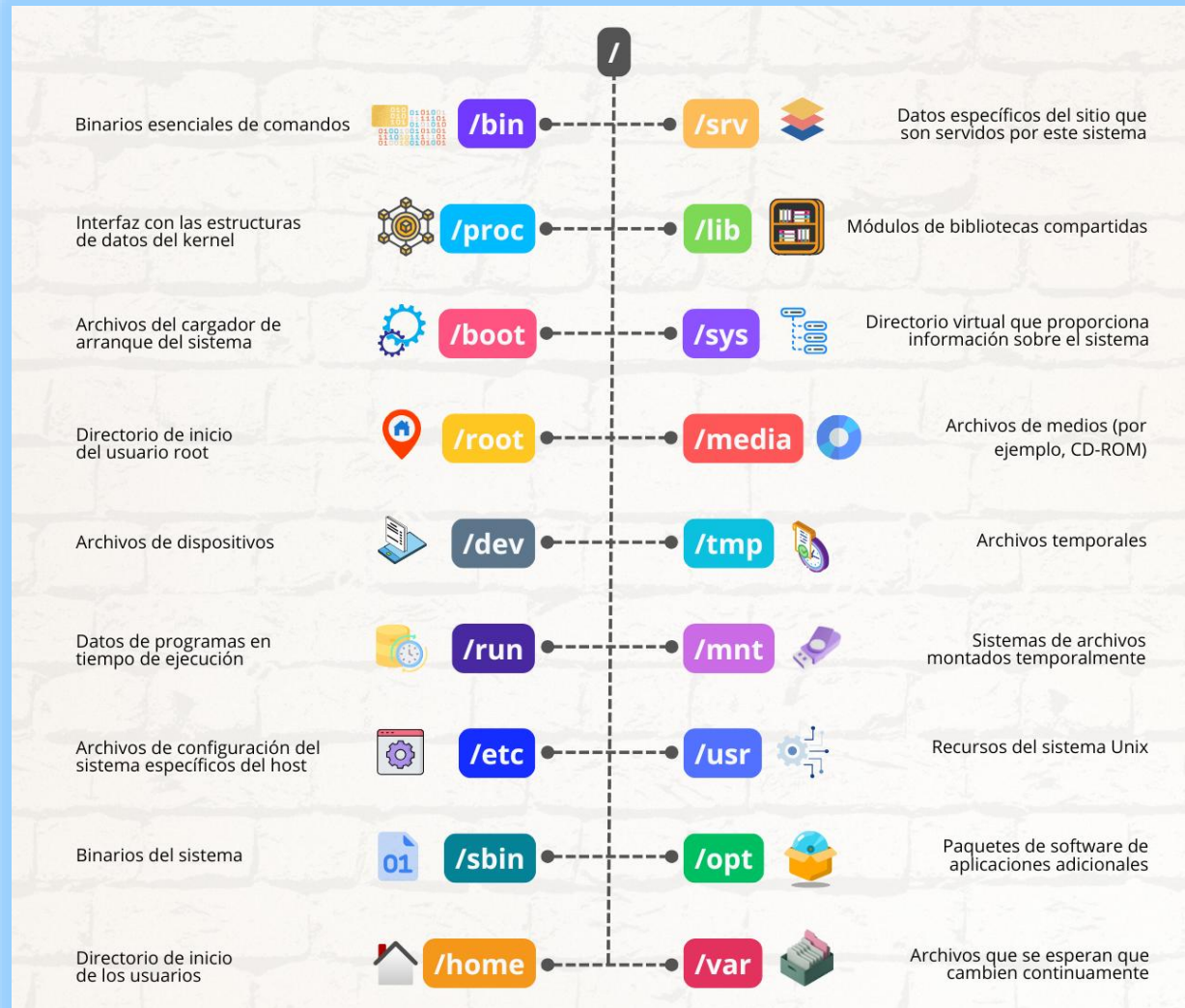
Ideas Fundamental

El significado de los datos es responsabilidad de la aplicación, no del sistema operativo

SISTEMA DE ARCHIVOS EN LINUX

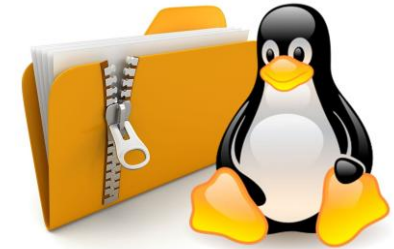
14

Organización Jerárquica



SISTEMA DE ARCHIVOS EN LINUX

15



Rutas absolutas y Relativas

Ruta absoluta: donde se indica cómo llegar al archivo desde el directorio raíz.

Ejemplos → `/usr/ast/libros/mos3/cap-10`

Ruta relativa: Los nombres de rutas también se pueden especificar en forma relativa al directorio

de trabajo

Ventajas:

- Menor longitud.
- Mayor legibilidad.
- Menor trabajo para el sistema.

Por ejemplo:

si `/usr/ast/libros/mos3` es el directorio de trabajo, entonces el comando del shell

`cp cap-10 respaldo-10`

tiene exactamente el mismo efecto que el siguiente comando más largo:

`cp /usr/ast/libros/mos3/cap-10 /usr/ast/libros/mos3/respaldo-10`

Directorio actual y directorio padre

Entradas especiales:

Entrada	Significado
<code>.</code>	Directorio actual
<code>..</code>	Directorio padre

Ejemplo:

`../lisa/x`

Uso frecuente:

- Navegación.
- Scripts.
- Administración.

SISTEMA DE ARCHIVOS EN LINUX

16

Enlaces simbólicos (Links)

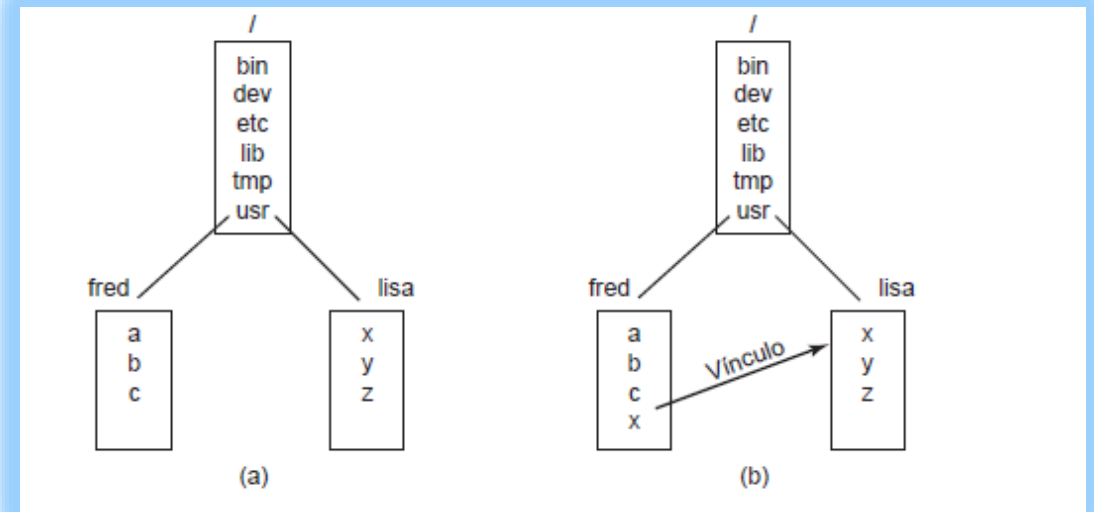
Con frecuencia ocurre que un usuario necesita hacer referencia a un archivo que pertenece a otro usuario, o que por lo menos se encuentra en alguna otra parte del archivo de directorios.



Objetivo: → Crear referencias alternativas a archivos existentes.

Conceptos:

- Archivo original.
- Nueva entrada de directorio.
- Compartición de recursos.



¿Por qué un enlace ocupa mucho menos espacio que copiar un archivo?

SISTEMA DE ARCHIVOS EN LINUX

17

Montaje de Archivos

Además de los archivos regulares, Linux también acepta los archivos especiales de caracteres y los archivos especiales de bloques. Los archivos especiales de caracteres se utilizan para modelar los dispositivos de E/S en serie, como los teclados y las impresoras. Al abrir y leer de `/dev/tty` se lee del teclado; al abrir y escribir en `/dev/lp` se escribe en la impresora. Los archivos especiales de bloques, que con frecuencia tienen nombres como `/dev/hd1`, se pueden utilizar para leer y escribir en particiones de disco crudas sin importar el sistema de archivos.



Problema:

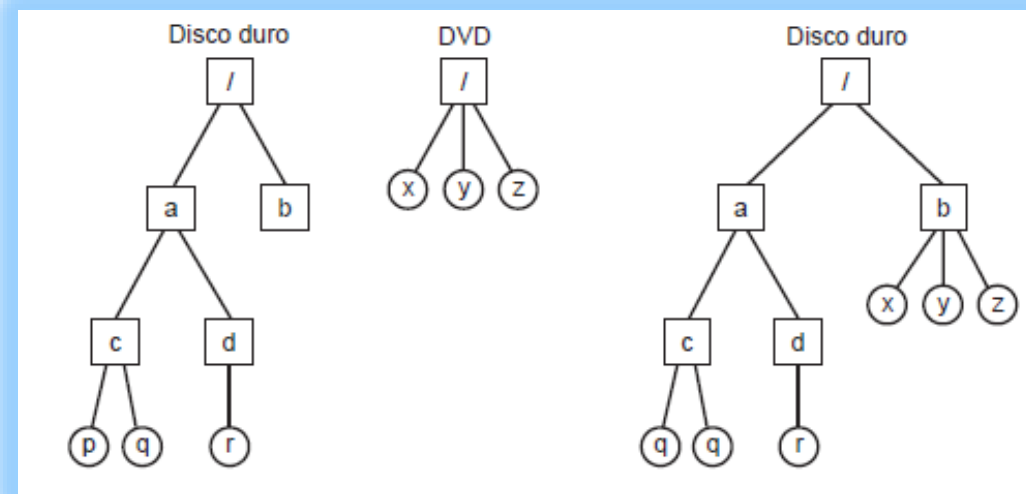
- Múltiples discos.

Solución Linux:

- Montar dispositivos dentro del árbol de directorios.

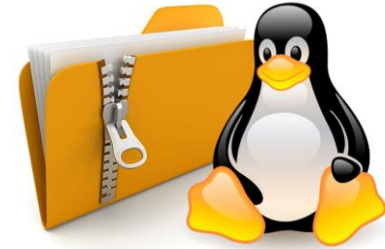
Concepto
Clave

Linux presenta un único árbol lógico.



SISTEMA DE ARCHIVOS EN LINUX

18



Bloqueo de Archivos (File Locking)

Necesidad:- → Evitar condiciones de carrera.

Tipos:

- Bloqueo compartido
- Bloqueo exclusivo

Aplicaciones:

- Bases de datos
- Archivos compartidos
- Sistemas multiusuario

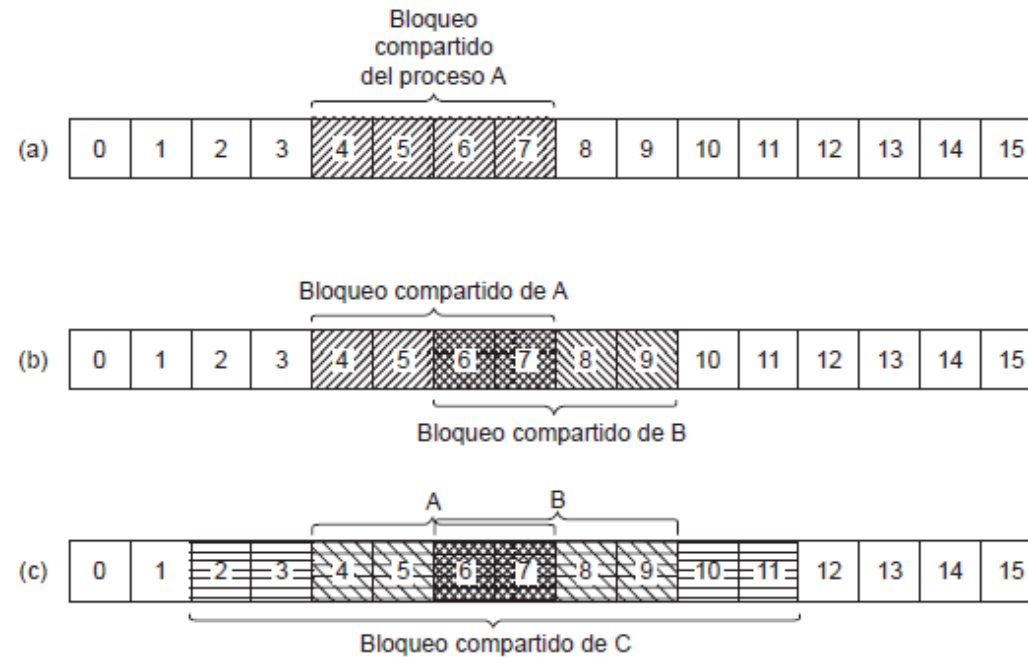


Figura 10-26. (a) Un archivo con un bloqueo. (b) Adición de un segundo bloqueo. (c) Un tercer bloqueo.

SISTEMA DE ARCHIVOS EN LINUX



Llamadas al sistema relacionadas con ARCHIVOS

Llamada al sistema	Descripción
da = creat(nombre, modo)	Una forma de crear un nuevo archivo
fd = open(archivo, como, ...)	Abre un archivo para lectura, escritura o ambas
s = close(fd)	Cierra un archivo abierto
n = read(fd, bufer, nbytes)	Lee datos de un archivo y los coloca en un búfer
n = write(fd, bufer, nbytes)	Escribe datos de un búfer a un archivo
posicion = lseek(fd, despl, de_donde)	Desplaza el apuntador del archivo
s = stat(nombre, &buf)	Obtiene la información de estado de un archivo
s = fstat(fd, &buf)	Obtiene la información de estado de un archivo
s = pipe(&fd[0])	Crea una tubería
s = fcntl(fd, cmd, ...)	Bloqueo de archivos y otras operaciones

Descriptores de archivo

Concepto: Número entero utilizado para identificar archivos abiertos.

Descriptores estándar:

FD	Uso
0	stdin
1	stdout
2	stderr

Ejemplo:

`sort <datos >resultado`

Ciclo de vida de un archivo

- creat()
- open()
- read()
- write()
- lseek()
- close()

Flujo completo:

Usuario → Kernel → Sistema de archivos → Disco

Dispositivo en el que está el archivo
Número de nodo-i (cuál archivo en el dispositivo)
Modo del archivo (incluye información de protección)
Número de vínculos al archivo
Identidad del propietario del archivo
Grupo al que pertenece el archivo
Tamaño del archivo (en bytes)
Hora de creación
Hora del último acceso
Hora de la última modificación

Stat

SISTEMA DE ARCHIVOS EN LINUX

Llamadas al sistema relacionadas con DIRECTORIOS

<code>s = mkdir(ruta, modo)</code>	Crea un nuevo directorio
<code>s = rmdir(ruta)</code>	Elimina un directorio
<code>s = link(rutaant, rutanueva)</code>	Crea un vínculo a un archivo existente
<code>s = unlink(ruta)</code>	Elimina el vínculo de un archivo
<code>s = chdir(ruta)</code>	Cambia el directorio de trabajo
<code>dir = opendir(ruta)</code>	Abre un directorio para leerlo
<code>s = closedir(dir)</code>	Cierra un directorio
<code>dirent = readdir(dir)</code>	Lee una entrada del directorio
<code>rewinddir(dir)</code>	Rebobina un directorio para poder volverlo a leer



SISTEMA DE ARCHIVOS EN LINUX

Implementación del Sistema de Archivos en LINUX

Sistema de Archivos Virtual (VFS)

Problema → Linux soporta múltiples sistemas de archivos.

Solución:

- VFS

Beneficios:

- Transparencia
- Portabilidad
- Extensibilidad



SISTEMA DE ARCHIVOS EN LINUX

Implementación del Sistema de Archivos en LINUX

Sistema de Archivos Virtual (VFS)

Objeto	Descripción
Superbloque	sistema de archivos específico
Dentry	entrada del directorio, un solo componente de una ruta
Nodo-i	archivo específico
Archivo	abre el archivo asociado con un proceso



Superbloque

- Información global del sistema.
- Cantidad de nodos-i.
- Cantidad de bloques.
- Metadatos críticos.

Dentry Caché

Objetivo: Acelerar búsquedas de rutas.

Ejemplo: /usr/ast/bin

La caché contiene:

- /
- /usr
- /usr/ast
- /usr/ast/bin

Ventaja: Reducción del acceso al disco.

Nodo - i

Contienen metadatos:

- Dueño
- Permisos
- Tamaño
- Fechas
- Ubicación de bloques
- ! El nombre del archivo NO está en el inodo.

SISTEMA DE ARCHIVOS EN LINUX

23

Sistema de Archivos EXT2

Estructura:

- Superbloque
- Mapa de bits
- Inodos

Características:

- Hasta 255 caracteres.
- Basado en inodos.

Sistema de Archivos EXT3

Agrega:

- ✓ Journaling (registro de transacciones)

Ventajas:

Menor pérdida de datos.

Recuperación más rápida.



SISTEMA DE ARCHIVOS EN LINUX

/proc

Sistema de archivos virtual.

Información del sistema:

- 1 /proc/cpuinfo
- 2 /proc/meminfo
- 3 /proc/PID

No existe físicamente en disco.

NFS

Permite compartir archivos entre computadoras.

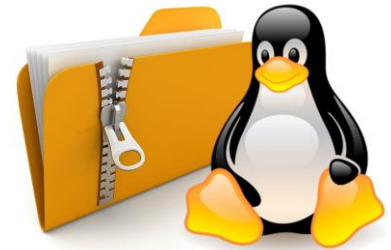
Conceptos:

- Cliente
- Servidor
- Directorios exportados
- Directorios montados



A NO OLVIDAR...

25



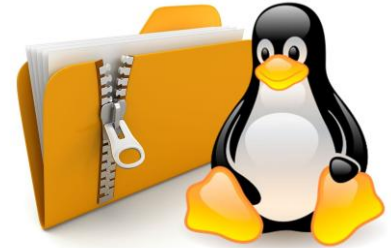
- ✓ Todo en Linux se maneja como archivo.
- ✓ Los dispositivos se representan en /dev.
- ✓ Los metadatos están en los inodos.
- ✓ El VFS permite múltiples sistemas de archivos.
- ✓ `ext3 = ext2 + journalin`

SISTEMA DE ARCHIVOS EN LINUX

26

Evolución de los Sistemas de Archivos

ext2 → ext3 → ext4



EXT2

- Sin journaling.
- Mayor rendimiento.
- Menor seguridad ante fallos.

EXT3

- Incorpora journaling.
- Recuperación más rápida.

EXT4

- Mayor escalabilidad.
- Mejor rendimiento.
- Mejor gestión del almacenamiento.

✓ Actualmente es el sistema de archivos Linux más utilizado.

SISTEMA DE ARCHIVOS EN LINUX



Características de EXT4

Reemplazan la lista tradicional de bloques.

Antes:

- 1 Bloque 10
- 2 Bloque 11
- 3 Bloque 12
- 4 Bloque 13
- 5 ...

Ahora:

- 1 Extensión:
- 2 Bloque 10 → Bloque 1000

Beneficios:

- Menos fragmentación.
- Menor uso de metadatos.
- Acceso más rápido.

Mejoras

Linux no asigna bloques inmediatamente.
((Delayed Allocation))

Ventajas:

- Mejor ubicación de datos.
- Menor fragmentación.
- Mejor rendimiento de escritura.

Característica	ext3	ext4
Tamaño máximo de archivo	2 TB	16 TB
Tamaño máximo sistema de archivos	16 TB	1 EB

Journaling mejorado

- Más eficiente.
- Menor sobrecarga.

Journaling mejorado

- Más eficiente.
- Menor sobrecarga.

Verificación más rápida (fsck)

Reduce significativamente el tiempo de comprobación del sistema de archivos.

SISTEMA DE ARCHIVOS EN LINUX

28

Comparativa EXT2, EXT3 Y EXT4



Característica	ext2	ext3	ext4
Journaling	✗	✓	✓
Extents	✗	✗	✓
Delayed Allocation	✗	✗	✓
Fragmentación	Alta	Media	Baja
Rendimiento	Bueno	Bueno	Muy bueno
Recuperación ante fallos	Baja	Alta	Muy alta